

Lista de implementação abordando **Interfaces** e o **Java Collections Framework**. Implemente na linguagem Java todos os exercícios propostos. O objetivo é aplicar os conceitos vistos em aula: definição e implementação de interfaces, polimorfismo, e o uso de List (ArrayList, LinkedList), Set (HashSet, TreeSet), Map (HashMap) e ordenação (Collections.sort).

Formato de Entrega

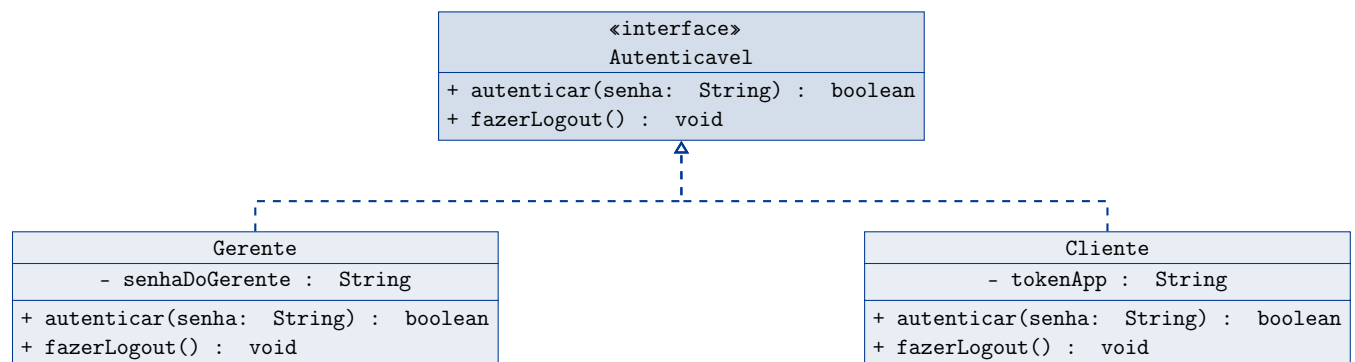
Instruções para Submissão

A entrega desta lista de exercícios deve ser feita através de um **único arquivo .zip**. Dentro deste arquivo, deve haver uma pasta estruturada para cada exercício (ex: Exercicio1, Exercicio2, ..., Exercicio6).

Atenção: Certifique-se de que cada pasta contenha **todos** os arquivos necessários (.java) para a correta compilação e execução do respectivo exercício.

1) Implementação de Interface e Polimorfismo

Modele um sistema simples de autenticação que demonstra o uso de interfaces e polimorfismo.

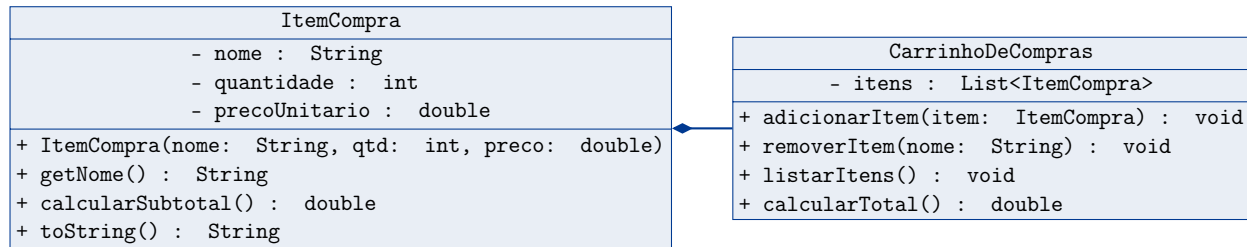


Requisitos

- Crie a interface **Autenticavel** com os métodos indicados no diagrama.
- Crie as classes **Gerente** e **Cliente** implementando a interface. Cada classe deve ter lógicas diferentes de autenticação (ex: o Gerente compara uma string fixa, o Cliente valida um token simulado).
- Crie uma classe **SistemaLogin** com um método `fazerLogin(Autenticavel usuario, String senha)` que recebe **qualquer** objeto que assine o contrato da interface.
- No Main, instancie gerentes e clientes, e passe-os para o método `fazerLogin` do sistema.

2) ArrayList e Manipulação de Elementos

Desenvolva um pequeno sistema orientado a objetos para gerenciar um carrinho de compras de um supermercado, utilizando a estrutura de dados `ArrayList`.

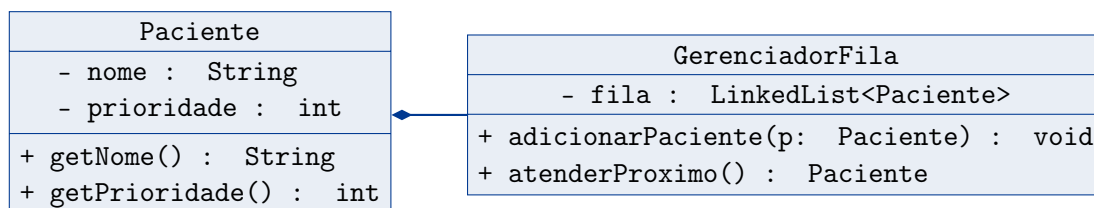


Requisitos

- Crie a classe `ItemCompra` devidamente encapsulada. O método `calcularSubtotal()` deve retornar a multiplicação da quantidade pelo preço unitário.
- Na classe `CarrinhoDeCompras`, o atributo `itens` deve ser declarado utilizando a interface `List` e instanciado como um `ArrayList`.
- Implemente `adicionarItem` utilizando o método `add()` da coleção.
- Implemente `removerItem` para buscar o item pelo nome e removê-lo. (Dica: pesquise sobre o método `removeIf()` da interface `Collection` ou utilize um `Iterator`).
- O método `listarItens` deve usar o laço `for-each` para percorrer a lista e exibir o `toString()` e o subtotal de cada item. O método `calcularTotal()` deve somar os subtotais de todos os itens do carrinho.
- Crie uma classe principal (`Main`) para instanciar o carrinho, adicionar alguns itens, remover pelo menos um deles e exibir o resumo final e o valor total.

3) LinkedList: Fila de Atendimento

Simule uma fila de atendimento de um hospital, onde a ordem de chegada dita a ordem de atendimento, mas emergências podem furar a fila.



Requisitos

- Utilize a classe `LinkedList` devido à sua eficiência em inserções e remoções no início/-meio.
- Crie um método `adicionarPaciente`. Se for uma emergência (prioridade alta), insira o paciente na **primeira posição** da lista (utilize os métodos específicos do `LinkedList` ou insira no índice 0). Caso contrário, insira no final.
- O método `atenderProximo()` deve remover e retornar o primeiro elemento da lista.

4) Trabalhando com Set (HashSet e TreeSet)

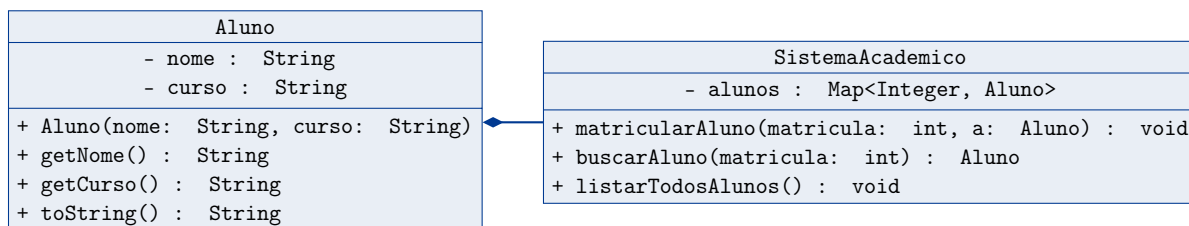
Implemente um sistema de registro de CPFs únicos para um evento.

Requisitos

- Primeiramente, utilize um `HashSet<String>` para tentar inserir diversos CPFs, incluindo alguns duplicados.
- Imprima a coleção e comprove que os duplicados foram ignorados e que a ordem de impressão pode ser aleatória.
- Em seguida, transfira todos os elementos do `HashSet` para um `TreeSet<String>`.
- Imprima o `TreeSet` e verifique se os CPFs foram automaticamente ordenados em ordem alfabética/numérica natural.

5) Map: Dicionário Chave-Valor (HashMap)

Desenvolva um sistema acadêmico orientado a objetos que mapeia números de matrícula para os registros completos dos alunos utilizando a estrutura `HashMap`.



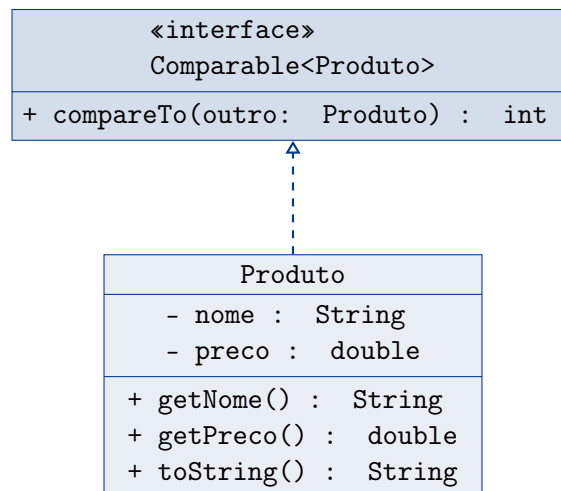
Requisitos

- Crie a classe `Aluno` com os atributos `nome` e `curso`, encapsulando-os corretamente e sobrescrevendo o método `toString()`.

- Na classe `SistemaAcademico`, instancie o atributo `alunos` utilizando a interface `Map` e a implementação `HashMap<Integer, Aluno>`. A chave será o número de matrícula (`Integer`).
- O método `matricularAluno` deve adicionar o par chave-valor usando `put()`. Se uma matrícula já existente for passada, permita a sobrescrita mas informe no console que o aluno foi atualizado.
- O método `buscarAluno` deve recuperar e retornar o aluno através do método `get(Object key)`, ou exibir uma mensagem se a matrícula não existir.
- O método `listarTodosAlunos` deve percorrer a coleção imprimindo as matrículas e os respectivos dados dos alunos. (Dica: utilize `keySet()` ou `entrySet()`).
- Na classe `Main`, instancie o sistema, matricule pelo menos 3 alunos, busque por uma matrícula específica e, em seguida, liste todos.

6) Ordenação Customizada com Comparable

Modele uma classe `Produto` e utilize a classe utilitária `Collections` para ordená-los.



Requisitos

- A classe `Produto` deve implementar a interface nativa do Java `Comparable<Produto>`.
- Sobrescreva o método `compareTo` para definir a ordem natural dos produtos baseada no **preço** (do menor para o maior).
- Crie um `ArrayList` de produtos e adicione vários objetos de forma desordenada.
- Utilize `Collections.sort(listaDeProdutos)` para ordenar a lista.
- Imprima a lista antes e depois da ordenação para validar a implementação. Dica: Sobrescreva o método `toString()` na classe `Produto` para facilitar a visualização.